Task 16: API Gateway to route client requests

Aim:

To implement an API Gateway that routes requests to multiple microservices and distributes load using round-robin load balancing.

Algorithm:

1. Create multiple instances of services (students, employees).
2. Store service URLs in arrays.
3. Maintain a counter for each service.
4. On each request:
    - Select next service using round robin.
    - Forward request using proxy middleware.
5. Return response to client.

Code:

Server.js

```
const express = require("express");

const bodyParser = require("body-parser");

const { createProxyMiddleware } = require("http-proxy-middleware");


/* ================================

  STUDENT SERVICE - INSTANCE 1

==================================*/

const studentService1 = express();

studentService1.use(bodyParser.json());


let students1 = [];


studentService1.get("/students", (req, res) => {

  res.json({ instance: "Student Service 1", data: students1 });

});


studentService1.post("/students", (req, res) => {
```

```
  students1.push(req.body);
  res.json({ msg: "Added in Service 1", data: students1 });
});

studentService1.put("/students/:id", (req, res) => {
  students1[req.params.id] = req.body;
  res.json({ msg: "Updated in Service 1", data: students1 });
});

studentService1.delete("/students/:id", (req, res) => {
  students1.splice(req.params.id, 1);
  res.json({ msg: "Deleted in Service 1", data: students1 });
});

studentService1.listen(3001, () =>
  console.log("Student Service 1 running on 3001")
);

/* ================================
   STUDENT SERVICE - INSTANCE 2
==================================*/
const studentService2 = express();
studentService2.use(bodyParser.json());

let students2 = [];

studentService2.get("/students", (req, res) => {
  res.json({ instance: "Student Service 2", data: students2 });
});
```

```
studentService2.post("/students", (req, res) => {
  students2.push(req.body);
  res.json({ msg: "Added in Service 2", data: students2 });
});

studentService2.put("/students/:id", (req, res) => {
  students2[req.params.id] = req.body;
  res.json({ msg: "Updated in Service 2", data: students2 });
});

studentService2.delete("/students/:id", (req, res) => {
  students2.splice(req.params.id, 1);
  res.json({ msg: "Deleted in Service 2", data: students2 });
});

studentService2.listen(3002, () =>
  console.log("Student Service 2 running on 3002")
);

/* ================================
   EMPLOYEE SERVICE - INSTANCE 1
==================================*/
const employeeService1 = express();
employeeService1.use(bodyParser.json());

let employees1 = [];

employeeService1.get("/employees", (req, res) => {
```

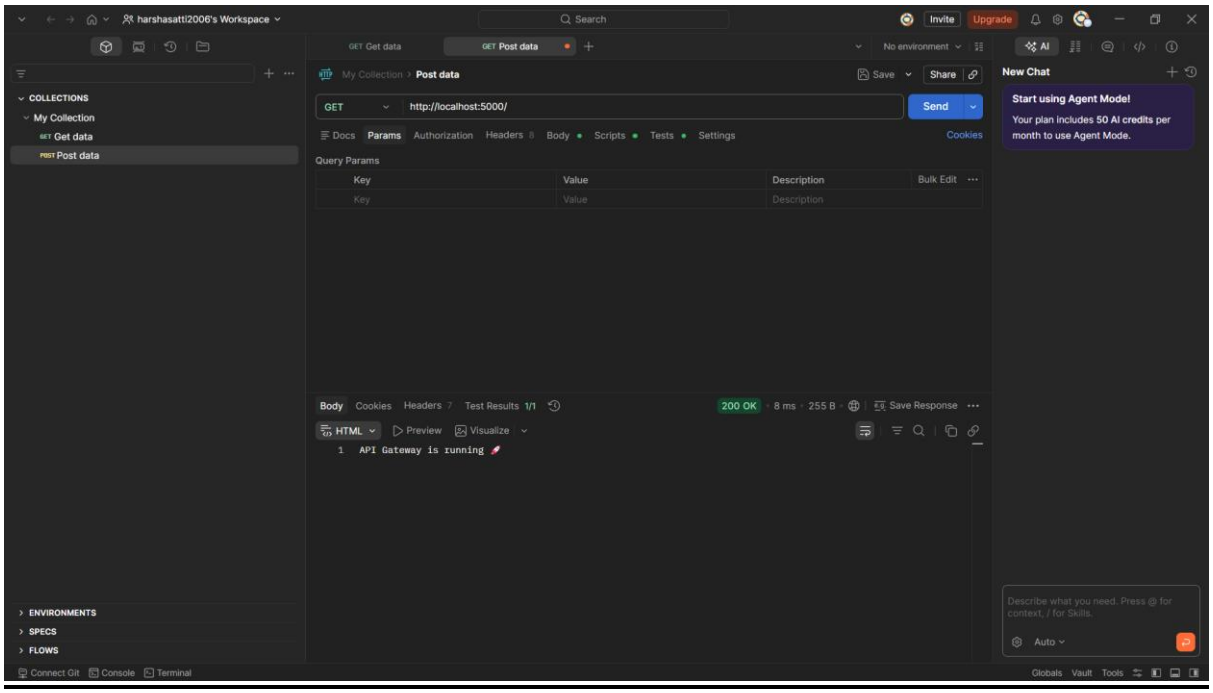
```
  res.json({ instance: "Employee Service 1", data: employees1 });
});

employeeService1.post("/employees", (req, res) => {
  employees1.push(req.body);
  res.json({ msg: "Added in Employee Service 1", data: employees1 });
});

employeeService1.put("/employees/:id", (req, res) => {
  employees1[req.params.id] = req.body;
  res.json({ msg: "Updated in Employee Service 1", data: employees1 });
});

employeeService1.delete("/employees/:id", (req, res) => {
  employees1.splice(req.params.id, 1);
  res.json({ msg: "Deleted in Employee Service 1", data: employees1 });
});

employeeService1.listen(4001, () =>
  console.log("Employee Service 1 running on 4001")
);

/* ================================
   EMPLOYEE SERVICE - INSTANCE 2
==================================*/
const employeeService2 = express();
employeeService2.use(bodyParser.json());

let employees2 = [];
```

```
employeeService2.get("/employees", (req, res) => {
  res.json({ instance: "Employee Service 2", data: employees2 });
});

employeeService2.post("/employees", (req, res) => {
  employees2.push(req.body);
  res.json({ msg: "Added in Employee Service 2", data: employees2 });
});

employeeService2.put("/employees/:id", (req, res) => {
  employees2[req.params.id] = req.body;
  res.json({ msg: "Updated in Employee Service 2", data: employees2 });
});

employeeService2.delete("/employees/:id", (req, res) => {
  employees2.splice(req.params.id, 1);
  res.json({ msg: "Deleted in Employee Service 2", data: employees2 });
});

employeeService2.listen(4002, () =>
  console.log("Employee Service 2 running on 4002")
);

/* ================================
   API GATEWAY
==================================*/
const app = express();
```

```
/* Default Route (Fix for your error) */
app.get("/", (req, res) => {
  res.send("API Gateway is running 🚀");
});

/* Round Robin Counters */
let studentCounter = { value: 0 };
let employeeCounter = { value: 0 };

/* Service URLs */
const studentServices = [
  "http://localhost:3001",
  "http://localhost:3002",
];

const employeeServices = [
  "http://localhost:4001",
  "http://localhost:4002",
];

/* Load Balancer */
function getNextService(services, counter) {
  const service = services[counter.value];
  counter.value = (counter.value + 1) % services.length;
  return service;
}

/* STUDENT ROUTE */
app.use("/students", (req, res, next) => {
```

```
  const target = getNextService(studentServices, studentCounter);
  console.log("Student request routed to:", target);

  createProxyMiddleware({
    target,
    changeOrigin: true,
  })(req, res, next);
});

/* EMPLOYEE ROUTE */
app.use("/employees", (req, res, next) => {
  const target = getNextService(employeeServices, employeeCounter);
  console.log("Employee request routed to:", target);

  createProxyMiddleware({
    target,
    changeOrigin: true,
  })(req, res, next);
});

/* START GATEWAY */
app.listen(5000, () => {
  console.log("API Gateway running at http://localhost:5000");
});
```

Outputs:



Result: Successfully implemented an API Gateway with dynamic routing and load balancing across multiple service instances. All CRUD operations work correctly via Postman.